Practical: Python GUI with MySQL Database

Question

Design and develop a Python GUI application using Tkinter to connect with a MySQL database. Implement `INSERT`, `SELECT`, `UPDATE`, and `DELETE` operations to store, retrieve, modify, and delete student records through the GUI.

Part A: Software Installation

Step 1: Install Python

If Python is not already installed:

1. Download and install Python 3.x.
2. During installation, **select "Add Python to PATH"**.
3. Complete the installation.

To verify Python installation, open **Command Prompt** and type:

```
python --version
```

Example output:

```
Python 3.13.5
```

Part B: Install MySQL

Step 2: Download MySQL Installer

Download **MySQL Installer for Windows** from the official MySQL website.

During installation, select:

MySQL Server
MySQL Workbench

MySQL Workbench will be useful for creating the database and executing SQL commands.

Step 3: Select Setup Type

When the MySQL Installer opens:

1. Select **Developer Default**.
2. Click **Next**.
3. Allow the installer to check the required components.
4. Click **Execute** to install the required components.

If you only need the database server and Workbench, you can also select the appropriate custom/server setup.

Step 4: Configure MySQL Server

During configuration:

Authentication

Use the default recommended authentication method unless your lab specifically requires another option.

MySQL Root Password

Set a password for the MySQL `root` user.

For example:

```
root123
```

Remember this password. It will be required in the Python program.

For classroom work, students can use their own password. Do not use a shared production password.

Step 5: Configure MySQL Server as a Windows Service

Keep the default service configuration.

For example:

```
Windows Service Name: MySQL80
```

Make sure the MySQL service is configured to start automatically.

Click **Next → Execute → Finish**.

Part C: Open MySQL Workbench

Step 6: Start MySQL Workbench

Open:

```
MySQL Workbench
```

You should see a connection such as:

```
Local instance MySQL80
```

Click the connection.

Enter the password created during MySQL installation.

Part D: Create Database

Step 7: Create Database

In MySQL Workbench, open a new SQL tab and execute:

```
CREATE DATABASE college_db;
```

Click the **Execute** button.

Step 8: Select the Database

Execute:

```
USE college_db;
```

Part E: Create Student Table

Step 9: Create Table

Execute:

```
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name VARCHAR(100),
    course VARCHAR(100),
    email VARCHAR(100)
);
```

Step 10: Verify the Table

Execute:

```
SHOW TABLES;
```

You should see:

```
students
```

Check the structure:

```
DESCRIBE students;
```

Part F: Install Python MySQL Connector

Step 11: Open Command Prompt

Open Command Prompt and execute:

```
pip install mysql-connector-python
```

Wait until the installation is completed.

To verify:

```
pip show mysql-connector-python
```

Part G: Create Python Project

Step 12: Create Project Folder

Create:

```
StudentMySQL
```

Inside the folder create:

```
student_mysql.py
```

Project structure:

```
StudentMySQL
│
└── student_mysql.py
```

Part H: Test MySQL Connection

Before creating the complete GUI, students should first test whether Python can connect to MySQL.

Add the following code to `student_mysql.py`:

```
import mysql.connector

connection = mysql.connector.connect(
    host="localhost",
    user="root",
    password="your_password",
    database="college_db"
)

if connection.is_connected():
    print("MySQL connection successful!")

connection.close()
```

Replace:

```
password="your_password"
```

with the password created during MySQL installation.

For example:

```
password="root123"
```

Run:

```
python student_mysql.py
```

Expected output:

```
MySQL connection successful!
```

Once this works, students can proceed with the GUI program.

Part I: Complete GUI Application

After successfully testing the connection, use the complete Tkinter program for the practical.

The application will contain:

```
Student Management System

Student ID:    [_____________]
Name:          [_____________]
Course:        [_____________]
Email:         [_____________]

[INSERT] [DISPLAY] [UPDATE]

[DELETE] [CLEAR]

--------------------------------
Student records will appear here
--------------------------------
```

The buttons perform:

Button	SQL Operation
INSERT	`INSERT`
DISPLAY	`SELECT`
UPDATE	`UPDATE`
DELETE	`DELETE`
CLEAR	Clears GUI fields

SQL Commands Used

INSERT

```
INSERT INTO students
(student_id, name, course, email)
VALUES (...);
```

SELECT

```
SELECT * FROM students;
```

UPDATE

```
UPDATE students
SET name = ..., course = ..., email = ...
WHERE student_id = ...;
```

DELETE

```
DELETE FROM students
WHERE student_id = ...;
```

Part J: Testing the Application

Test 1 – Insert

Enter:

```
Student ID: 101
Name: Rahul
Course: BSc Computer Applications
Email: rahul@gmail.com
```

Click:

INSERT

Expected message:

```
Student record inserted successfully.
```

Test 2 – Retrieve

Click:

DISPLAY

The record should be displayed in the GUI.

Test 3 – Update

Enter:

```
Student ID: 101
Name: Rahul Sharma
Course: BSc Computer Applications
Email: rahulsharma@gmail.com
```

Click:

UPDATE

Expected message:

```
Student record updated successfully.
```

Test 4 – Delete

Enter:

```
Student ID: 101
```

Click:

DELETE

Expected message:

```
Student record deleted successfully.
```

Test 5 – Verify Database

Open MySQL Workbench and execute:

```
USE college_db;

SELECT * FROM students;
```

Students can verify that the changes made through the **Python GUI are reflected in the MySQL database**.

Final Practical Flow

```
Install Python
       ↓
Install MySQL Server + Workbench
       ↓
Configure MySQL
       ↓
Create college_db
       ↓
Create students table
       ↓
Install mysql-connector-python
       ↓
Test Python–MySQL Connection
       ↓
Create Tkinter GUI
       ↓
INSERT
       ↓
SELECT
```

```
        ↓
UPDATE
        ↓
DELETE
        ↓
Verify in MySQL Workbench
```

This version is suitable to give directly to students as a **step-by-step practical sheet**, because it includes the setup, database creation, connector installation, connection test, GUI implementation, and testing.